Task 3. Technical mini-project (50 points).

Quest: "Development of a phishing campaign simulation system"

Introduction

Imagine that you have joined a company's information security team.

You are tasked with developing a small demonstration stand that allows for conducting educational phishing campaigns within the organization.

The system must be able to:

- send educational emails to users;
- track link clicks;
- save information about events;
- be launched with a single command via Docker Compose.

Task 1. Prepare the project infrastructure

First of all, it is necessary to prepare the working environment.

Deploy the project using **Docker Compose** so that the entire system can be launched with a single command:

docker compose up -d

The project must include the following ingredients:

- Python application (Tracker);
- Nginx;
- Mail Service;
- SQLite database.

After launch, all services must start without errors and be ready for operation.

What will be checked

- correctness of docker-compose.yml;
- project structure;
- successful container launch;
- absence of errors in the logs.

Task 2. Prepare an educational page

Imagine that a user has received an email and clicked on the link.

Upon opening the link, they should see a page with a notification that this was an educational phishing campaign.

Configure **Nginx** to work via port **8084** and host an HTML page with the following text:

You have clicked on an educational phishing link.

If this email has caused suspicion, please report it to the information security department.

For example, after clicking the link

http://localhost:8084/?email=user@test.ru&token=123456

the user should see this page.

What will be checked

- Nginx operation;
- correct handling of HTTP requests;
- page display.

Task 3. Teach the system to record clicks

Now it is necessary to ensure that every user click is saved in the database.

When opening the link, the system must identify and save:

- user email;
- unique token;
- IP address;
- date and time of the click.

For example,

http://localhost:8084/?email=david@test.ru&token=8f34d2

an entry of approximately the following type should appear in the database:

Email	Token	IP	Date
david@test.ru	8f34d2	10.10.10.15	2026-08-15 10:15

You must use **SQLite**.

What will be checked:

- correctness of the database structure;
- data storage;
- handling URL parameters;
- code readability.

Task 4. Implement an email sending service

The next stage is automating the mailing list.

You need to write a Python service that:

- reads a list of users from a CSV file;
- generates a unique token;
- creates a personalized link;
- sends an email via SMTP.

For example, if the CSV contains a user

davir@test.ru

then the email can contain a link like

http://localhost:8084/?email=david@test.ru&token=8f34d2...

To demonstrate the email, you need to send an email to the address

shiryaev_a@goldapple.ru

You can use any SMTP service.

What will be checked:

- reading CSV;
- generation of unique links;
- sending emails;
- correctness of the project structure.

Task 5. Prepare the project for submission

Format the results of your work as a public GitHub repository.

The minimal project structure should look like this:

project/

docker-compose.yml

README.md

.gitignore

tracker/

mailer/

nginx/

docs/

The README should contain a brief instruction on how to run the project.

What will be checked:

- repository structure;
- clarity of the README;
- ability to quickly run the project.

Task 6. Prepare documentation

After completing the development, you need to prepare a short report in **Word** or **PDF** format.

The report must contain:

- run instructions;
- a link to GitHub;
- a link to a video demonstration (a short 30-60 second clip showing: docker run -> sending an email -> opening the link on localhost -> recording the link opening in the DB -> notification in Telegram (optional));
- screenshots of the main implementation stages with brief comments.

The video must demonstrate:

- running Docker Compose;
- sending an email;
- clicking the link;
- opening the page;
- the appearance of a record in the database.

Additional task. Integration with Telegram

If the main part of the project is completed, implement notifications in Telegram.

After each user transition, the bot must automatically send a message in the following format:

Phishing transition

Email:

ivan@test.ru

IP:

10.10.10.15

Date:

2026-08-15 10:15

Token:

8f34d2...

This task is not mandatory, but it will allow you to earn extra points and demonstrate your skills in working with external APIs.
You can connect my telegram: 396538835

Implementation recommendations:

The use of any AI tools (ChatGPT, GitHub Copilot, Claude, etc.) is permitted. What is evaluated is not the fact of using AI, but the ability to independently build a working system, understand emerging errors, and explain the technical decisions made.